

Final Model Evaluation & Interpretation

Goal

Evaluate the final selected models on an unseen test set and interpret their predictive performance. This notebook focuses on model evaluation, error analysis, and interpretability, not model training.

```
In [1]: import pandas as pd
import numpy as np
from pathlib import Path

from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    roc_auc_score,
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay
)
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression

import matplotlib.pyplot as plt
```

```
In [2]: PROJECT_ROOT = Path.cwd()
DATA_PATH = PROJECT_ROOT / "Data"

df = pd.read_csv(DATA_PATH / "ObesityDataSet_raw_and_data_synthetic.csv")
```

```
In [3]: # Binary target
df["obesity_risk"] = df["NObeyesdad"].apply(
    lambda x: 0 if x in ["Insufficient_Weight", "Normal_Weight"] else 1
)

y = df["obesity_risk"]

X = df.drop(
    columns=[
        "NObeyesdad",
        "obesity_risk",
        "Height",
        "Weight"
    ]
)
```

```
In [4]: categorical_columns = X.select_dtypes(include="object").columns

X_encoded = pd.get_dummies(
    X,
    columns=categorical_columns,
```

```
    drop_first=True
)

X_encoded = X_encoded.reset_index(drop=True)
y = y.reset_index(drop=True)
```

Train / Test Split

```
In [5]: X_train, X_test, y_train, y_test = train_test_split(
        X_encoded,
        y,
        test_size=0.2,
        stratify=y,
        random_state=42
    )
```

Random Forest

```
In [6]: rf_final = RandomForestClassifier(
        n_estimators=100,
        max_depth=20,
        min_samples_split=2,
        min_samples_leaf=1,
        random_state=42
    )

rf_final.fit(X_train, y_train)
```

```
Out[6]: ▼ RandomForestClassifier ⓘ ?
        ► Parameters
```

Evaluation — Random Forest

```
In [7]: rf_proba = rf_final.predict_proba(X_test)[: , 1]
        rf_pred = rf_final.predict(X_test)

        print("=== Random Forest - Classification Report ===")
        print(classification_report(y_test, rf_pred))

        print("ROC-AUC:", roc_auc_score(y_test, rf_proba))
```

```
=== Random Forest – Classification Report ===
              precision    recall  f1-score   support

     0       0.87        0.89        0.88        112
     1       0.96        0.95        0.96        311

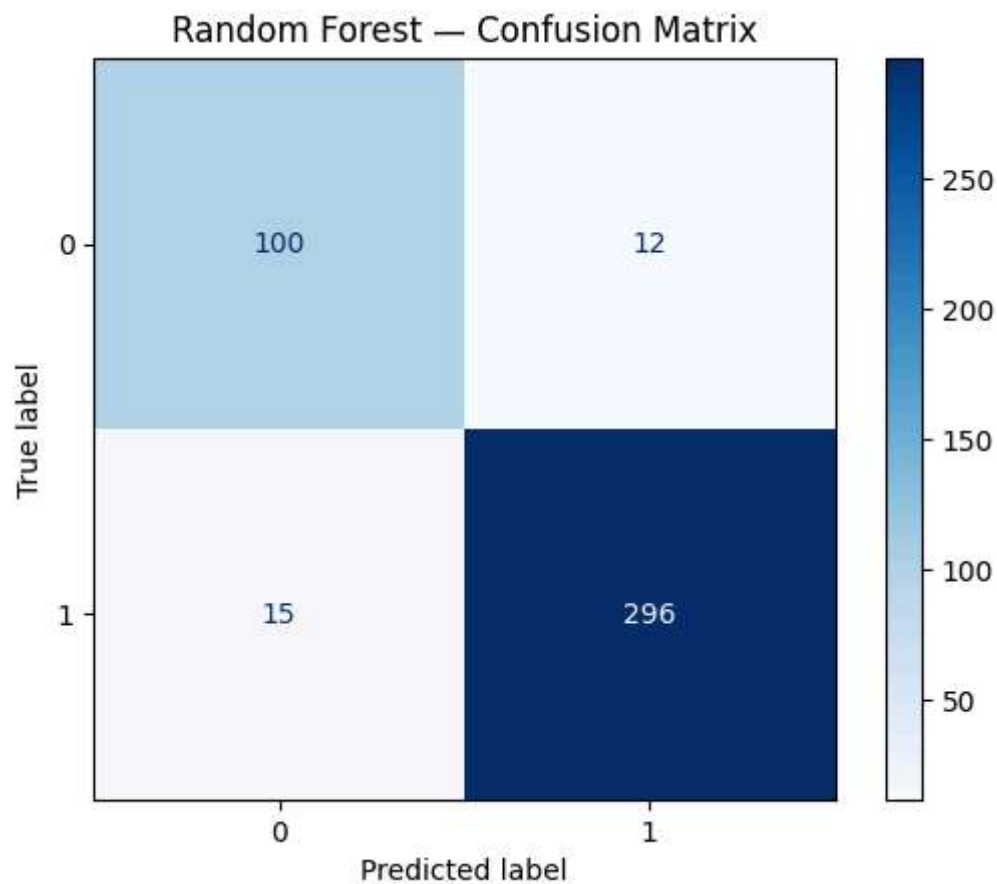
 accuracy          0.94          423
 macro avg         0.92          423
 weighted avg      0.94          423
```

ROC-AUC: 0.9794011254019294

Confusion Matrix — Random Forest

```
In [8]: ConfusionMatrixDisplay.from_predictions(
        y_test,
        rf_pred,
        cmap="Blues",
        values_format="d"
    )

plt.title("Random Forest – Confusion Matrix")
plt.show()
```



Logistic Regression

```
In [9]: logreg_final = LogisticRegression(
    C=1,
    penalty="l1",
    class_weight="balanced",
    solver="liblinear",
    max_iter=1000,
    random_state=42
)

logreg_final.fit(X_train, y_train)
```

C:\Users\biauser\anaconda3\envs\edal\Lib\site-packages\sklearn\linear_model_logistic.py:1135: FutureWarning: 'penalty' was deprecated in version 1.8 and will be removed in 1.10. To avoid this warning, leave 'penalty' set to its default value and use 'l1_ratio' or 'C' instead. Use l1_ratio=0 instead of penalty='l2', l1_ratio=1 instead of penalty='l1', and C=np.inf instead of penalty=None.

warnings.warn(

C:\Users\biauser\anaconda3\envs\edal\Lib\site-packages\sklearn\linear_model_logistic.py:1160: UserWarning: Inconsistent values: penalty=l1 with l1_ratio=0.0. penalty is deprecated. Please use l1_ratio only.

warnings.warn(

Out[9]: **LogisticRegression** ⓘ ?

► Parameters

Evaluation - Logistic Regression

```
In [10]: logreg_proba = logreg_final.predict_proba(X_test)[: , 1]
logreg_pred = logreg_final.predict(X_test)

print("=== Logistic Regression - Classification Report ===")
print(classification_report(y_test, logreg_pred))

print("ROC-AUC:", roc_auc_score(y_test, logreg_proba))
```

```
=== Logistic Regression - Classification Report ===
              precision    recall  f1-score   support

     0              0.75         0.85         0.80         112
     1              0.94         0.90         0.92         311

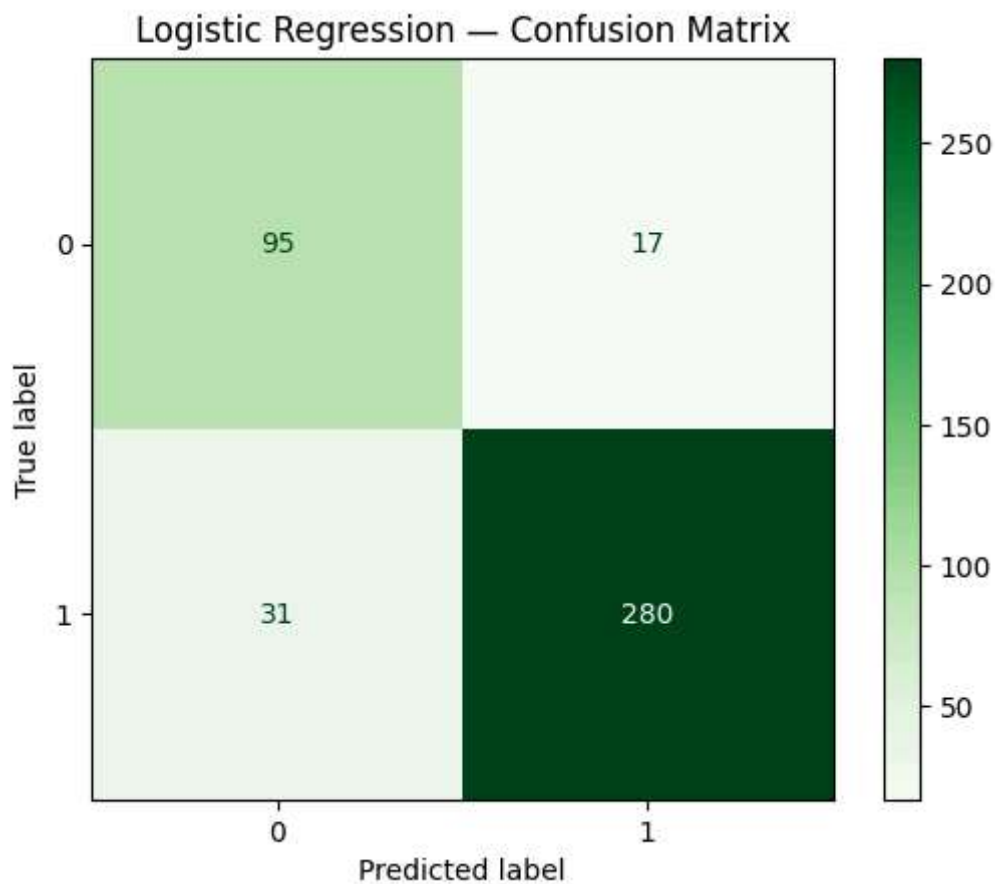
 accuracy              0.89         0.89         0.89         423
 macro avg              0.85         0.87         0.86         423
weighted avg              0.89         0.89         0.89         423

ROC-AUC: 0.9246095544327056
```

Confusion Matrix — Logistic Regression

```
In [11]: ConfusionMatrixDisplay.from_predictions(
    y_test,
    logreg_pred,
```

```
cmap="Greens",  
values_format="d"  
)  
  
plt.title("Logistic Regression — Confusion Matrix")  
plt.show()
```



FINAL — Interpretation

Final Evaluation Summary

Random Forest achieved higher overall predictive performance, reflected in ROC-AUC and recall for the risk class. Logistic Regression showed slightly lower performance but offered clearer interpretability and more transparent decision boundaries.

In risk-screening contexts, reducing false negatives is critical. While Random Forest performs strongly, Logistic Regression remains a viable alternative when interpretability is prioritized.